Contributing to NEXUS-R

> We build through specification, isolation, and verification.

Development Philosophy

NEXUS-R follows an **agent-native development methodology** derived from the reality that coding agents write 100% of the code. This inverts traditional engineering:

- **Specification is the product** — Every interface, type, error code, and test scenario must be defined before implementation.
- **Agents work in isolation** — Each agent owns one module with zero ambiguity about boundaries.
- **Verification precedes integration** — No module enters the codebase without passing a test suite that a second agent validates.
- **Event-source the build process** — Every agent decision, test result, and integration outcome is logged.

If you are a human contributor, you are the **Meta-Agent**: you route tasks, feed context, and apply patches. If you are an agent contributor, you work from `.spec.md` files and never modify specs.

Repository Structure

```
nexus-r/
├── specs/                    # Source of truth. Read, never write.
│   ├── 00_architecture.md
│   ├── 01_input_gateway.spec.md
│   ├── 02_cognition_router.spec.md
│   ├── 03_execution_sandbox.spec.md
│   ├── 04_state_core.spec.md
│   ├── 05_workflow_engine.spec.md
│   ├── 06_trust_layer.spec.md
│   └── 99_acceptance_criteria.md
├── foundation/nexus_r/       # Shared infrastructure
│   ├── events.py             # EventStore, CausalEvent
│   ├── models.py             # ModelRouter, LiteLLM wrapper
│   ├── config.py             # NEXUSConfig, Pydantic settings
│   ├── exceptions.py         # Exception hierarchy
│   └── telemetry.py          # Structured logging, spans
├── modules/
│   ├── input_gateway/        # Intent parsing, classification
│   ├── cognition_router/     # CAR, provider profiles, fallback
│   ├── execution_sandbox/    # MCP tools, sandbox, verification
│   ├── state_core/           # EventStore, WorkingState, Identity
│   ├── workflow_engine/      # Trace recording, ETD (Phase 2+)
│   ├── trust_layer/          # Permissions, audit, secrets
│   ├── session_manager/      # Session lifecycle, recovery
│   └── cli/                  # Typer CLI interface
├── tests/
│   ├── unit/                 # Per-module tests (≥90% target)
│   ├── integration/          # Cross-module pipeline tests
```

Generated by Kimi.ai

```
│   ├── security/             # Adversarial and hardening tests
│   └── fixtures/             # Shared test data
├── scripts/
│   ├── verify_module.py      # CLI: run tests + coverage for one module
│   ├── phase1_benchmark.py   # Performance benchmarks
│   └── phase1_failure_injection.py  # Chaos engineering
├── docs/                     # Architecture reviews, audit reports
├── e2e/                      # End-to-end CLI tests
└── pyproject.toml
```

Every module directory MUST contain:

- `README.md` — 10-line module contract
- `src/` — Implementation
- `tests/` — Unit tests runnable via `pytest` independently
- `interface.yaml` — Declared dependencies on other modules and external packages

---

Setup

Prerequisites

- Python 3.12+
- Git
- Ollama (for local model validation)

Install

```
git clone https://github.com/yourorg/nexus-r.git
cd nexus-r
python -m venv .venv
source .venv/bin/activate  # Windows: .venv\Scripts\activate
pip install -e ".[dev]"
```

Verify

```
pytest tests/unit/ -v                    # Unit tests
pytest tests/integration/ -v              # Integration tests
pytest tests/security/ -v                 # Security tests
pytest --cov=nexus_r --cov-report=term    # Coverage check
```

Testing Standards

Coverage

- **Minimum per module:** 80%
- **Target per module:** 90%
- **Security tests:** Mandatory for every phase

Test Categories

Category	Purpose	Location
Unit	Module correctness in isolation	`tests/unit/test_{module}.py`
Adversarial	Injection, traversal, spoofing	`tests/unit/test_{module}_adversarial.py`
Integration	Cross-module pipeline	`tests/integration/test_*.py`
Security	Vulnerability hardening	`tests/security/test_*_hardening.py`
Stress	Concurrency, memory, load	`tests/stress/test_*.py`
Failure Injection	Crash, network, corruption	`tests/failure/test_*.py`

Writing Adversarial Tests

Every module must have adversarial tests covering:

- Empty inputs, oversized inputs, unicode, random bytes
- Injection attacks (SQL, command, path traversal, prompt)
- Concurrency races
- Resource exhaustion (disk full, memory limit)

Example pattern:

```
def test_parse_never_crashes():
    # Property: For any input, parse() returns valid IntentResult
    for fuzz_input in generate_fuzz_inputs(count=1000):
        result = parser.parse(fuzz_input)
        assert isinstance(result, IntentResult)
        assert 0.0 <= result.confidence <= 1.0
```

Agent-Native Contribution Flow

If you are contributing via coding agent (Codex, Claude Code, etc.):

1. **Read the spec** — Start with `specs/00_architecture.md` and the relevant `.spec.md` file.
2. **Do not modify specs** — Specs are immutable contracts. If you find a bug in a spec, open a human-reviewed issue.
3. **Work in isolation** — Only read your module's `interface.yaml` and the foundation code. Do not read other modules' implementations.
4. **Write tests first** — Adversarial and property-based tests must accompany every change.
5. **Pass security audit** — A second agent (or human) must review security implications before merge.

Code Style

- **Formatter:** `ruff`

Generated by Kimi.ai

- **Type hints:** Mandatory on all public functions
- **Docstrings:** Google style
- **Async:** Prefer `async`/`await` for I/O-bound operations
- **Error codes:** Every module has a reserved range (e.g., IG-001 to IG-050 for Input Gateway)

Security

See `security_audit_phase1.md` for current posture.

Rules

- Secrets never appear in logs, ETD records, or model context
- Sandbox cannot escape workspace boundary (verified in CI)
- Audit log is append-only — no deletion or modification paths
- T4 actions require explicit confirmation with documented reason

Reporting Vulnerabilities

Email security@nexus-r.dev with:

- Affected component and version
- Reproduction steps
- Impact assessment
- Suggested fix (if any)

Benchmarks

Before submitting changes that affect performance:

```
python scripts/phase1_benchmark.py
```

Compare against baseline in `benchmark_report_phase1.md`.

Questions?

- Open a Discussion for architecture questions
- Open an Issue for bugs or feature requests
- Tag with `phase-1`, `phase-2`, `security`, `performance`, or `documentation`

> Specification is the product. Verification precedes integration.

Generated by Kimi.ai